

Accelerating Number Theoretic Transform with Multi-GPU Systems for Efficient Zero Knowledge Proof

Zhuoran Ji*
School of Cyber Science and
Technology
Shandong University
Qingdao, Shandong, China
zrji@sdu.edu.cn

Jianyu Zhao
School of Cyber Science and
Technology
Shandong University
Qingdao, Shandong, China
201400620165@mail.sdu.edu.cn

Peimin Gao
School of Cyber Science and
Technology
Shandong University
Qingdao, Shandong, China
202100460055@mail.sdu.edu.cn

Xiangkai Yin*
School of Cyber Science and
Technology
Shandong University
Qingdao, Shandong, China
yinxiangkai@mail.sdu.edu.cn

Lei Ju
Quan Cheng Laboratory
Jinan, Shandong, China
Corresponding Author ✉
sr-julei@qcl.edu.cn

Abstract

Zero-knowledge proofs validate statements without revealing any information, pivotal for applications such as verifiable outsourcing and digital currencies. However, their broad adoption is limited by the prolonged proof generation times, mainly due to two operations: Multi-Scalar Multiplication (MSM) and Number Theoretic Transform (NTT). While MSM has been efficiently accelerated using multi-GPU systems, NTT has not, due to the high inter-GPU communication overhead incurred by its permutation data access pattern.

This paper identifies the necessity of multi-GPU NTT support for end-to-end proof generation. It introduces *UniNTT*, an NTT algorithm tailored for multi-GPU systems. The data access pattern of NTT incurs communication across all levels of the multi-GPU hierarchy (i.e., warp, thread block, GPU, and multi-GPU), complicating the implementation of multi-GPU NTT. To this end, *UniNTT* proposes a novel, overhead-free decomposition approach that recursively decomposes an NTT into smaller NTTs, enabling all hierarchy levels execute the same NTT computations at different scales. It promotes a uniform design of NTT optimizations based on an abstract

hardware model, which are then tailored and applied to different levels of the hierarchy. *UniNTT* not only simplifies the optimization process but also shows that optimizations typically specific to one level can also be effectively generalized to others. Experiments show that *UniNTT* achieves an average 4.26× speedup compared to leading NTT implementations when both are executed on an 8-GPU system.

CCS Concepts: • Computing methodologies → Massively parallel algorithms; • Security and privacy;

Keywords: Zero knowledge proof; number theoretic transform; multi-GPU systems

ACM Reference Format:

Zhuoran Ji, Jianyu Zhao, Peimin Gao, Xiangkai Yin, and Lei Ju. 2025. Accelerating Number Theoretic Transform with Multi-GPU Systems for Efficient Zero Knowledge Proof. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25)*, March 30–April 3, 2025, Rotterdam, Netherlands. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3669940.3707241>

1 Introduction

Zero-knowledge proofs (ZKPs) are cryptographic primitives that enable one party to prove a statement without revealing any information [18, 19]. For instance, users can verify access to a restricted website without exposing their identity. Among ZKP protocols, *zkSNARK* (Zero-Knowledge Succinct Non-interactive Argument of Knowledge) stands out by ensuring constant size and verification time for any proof [17, 18, 20, 22]. Thus, *zkSNARK* has gained attention in various fields such as electronic voting [21], verifiable machine learning [3, 11], and digital currencies [7, 8, 12, 13].

While *zkSNARK* offers fast verification, its widespread adoption is limited by extensive proof generation times [30–32, 40, 41]. Even with GPU acceleration, proof generation can still take several minutes [12, 13], prompting the use of

*Also with Quan Cheng Laboratory, Jinan, Shandong, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. *ASPLOS '25, March 30–April 3, 2025, Rotterdam, Netherlands*

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0698-1/25/03

<https://doi.org/10.1145/3669940.3707241>

multiple GPUs for further acceleration. The core computations involve multi-scalar multiplication (MSM) and number-theoretic transform (NTT). MSM, benefiting from minimal internal data dependencies, scales nearly linearly with the number of GPUs [25, 30, 31]. In contrast, NTT exhibits a permutation data access pattern that requires inter-GPU communication, complicating its multi-GPU implementation. As a result, prior studies have mainly focused on multi-GPU implementations for MSM, with fewer exploring NTT.

However, multi-GPU support remains crucial for NTT in ZKP. The outputs of NTT serve as inputs for MSM. If MSM employs multi-GPU acceleration, the NTT outputs must be distributed across the GPUs. This introduces inevitable communication overhead, regardless of whether NTT is executed on one or multiple GPUs. Thus, counterintuitively, multi-GPU NTT actually reduces communication time in end-to-end proof generation. This efficiency gain occurs because the total communication volume remains constant while the number of involved GPUs increases.

Designing an efficient multi-GPU NTT algorithm is challenging due to the complex hierarchical structure, which includes threads, warps, thread blocks, single GPUs, and multi-GPU systems. Although threads are the fundamental units in GPU programming, the higher levels are not transparent in NTT implementations. The permutation data access pattern in NTT necessitates data exchanges across all levels, each with distinct memory resources and execution modes. Thus, optimizations must be tailored for each NTT round and every level of the multi-GPU hierarchy.

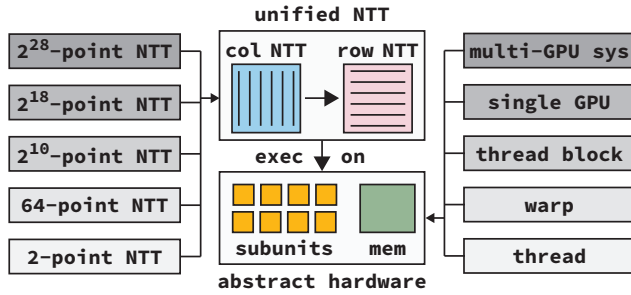


Figure 1. Systematically deploy NTT to multi-GPU system

To this end, this paper presents *UniNTT*, an multi-GPU NTT algorithm optimized through a unified approach and systematically mapped onto multi-GPU systems. As illustrated in Figure 1, it recursively decomposes an NTT into smaller NTTs, allowing all levels of the multi-GPU hierarchy to perform the same NTT computations at different scales. *UniNTT* enables NTT optimizations to be designed based on a unified abstract hardware model, which is then tailored and applied across various levels of the multi-GPU hierarchy. Our approach innovates on three fronts:

In decomposing an NTT, *UniNTT* arranges the N input data into an $N = N_1 \times N_2$ matrix, illustrating that an NTT

can be decomposed into N_2 N_1 -point NTTs and N_1 N_2 -point NTTs, with a matrix transpose in between. Unlike the 4-step NTT, which requires element-wise multiplication after the transpose, *UniNTT* optimizes for GPUs by using distinct primitive roots in each NTT in the second stage, thereby eliminating element-wise multiplication. It removes decomposition overhead and enables aggressive NTT decomposition at each level of the multi-GPU hierarchy. Additionally, *UniNTT* introduces a flexible decomposition approach that combines standard NTT rounds with decomposition to enable further optimizations.

As each level of the multi-GPU hierarchy executes the same NTT computations of varying sizes, *UniNTT* introduces an abstract hardware model that simplifies the architecture into two components: subunits for NTT execution and memory for data exchange, without specifying actual hardware types. This abstraction enables unified analysis and optimization of NTT, applicable to both a single warp and a multi-GPU system. *UniNTT* employs innovative strategies including customized decomposition, primitive root reuse and prefetch, input data layout optimization, multiplication elimination, and pipelining.

These optimizations are then systematically tailored and applied across different levels of the multi-GPU hierarchy. Notably, optimizations initially designed for one level often prove beneficial at others. For example, the pipelining method, originally intended for inter-GPU communication, also enhances memory latency hiding on single GPUs and even within thread blocks. Additionally, though thread blocks were once the finest granularity for NTT decomposition, allowing each warp to execute an NTT also improves performance by reducing data exchanges.

This work makes the following major contributions:

- We identify the importance of multi-GPU support for NTT in ZKP. Experiments show that *UniNTT* achieves an average $4.26\times$ speedup compared to SOTA.
- We propose an overhead-free decomposition method that enables aggressive decomposition to align NTT computations with the multi-GPU hierarchy.
- We introduce an abstract hardware model for unified analysis and optimization of NTT, which are then systematically applied to each level of the multi-GPU hierarchy to achieve comprehensive optimization.

Although this work primarily aims to accelerate NTT in ZKP, the proposed optimization routine may also facilitate architecture-aware optimizations for other algorithms on multi-GPU systems.

2 Background

2.1 Applications of zkSNARK

ZKP is a powerful cryptographic primitive [18, 19]. At its core, a prover can convince a verifier that, “I know a secret witness w for a function F and input x , such that $F(x, w) = 0$ ”,

without disclosing the value of w . Among various ZKP protocols, *zkSNARKs* [17, 18, 22] stand out due to their distinct features: (1) succinctness, with proof sizes under 1KB and verification times below 10ms; (2) non-interactivity, requiring only a single message from the prover to verifiers; (3) zero-knowledge, proof is verified without disclosing w .

With these properties, *zkSNARK* has been adopted in various fields, including electronic voting [21], verifiable machine learning [3, 11], and digital currencies [7, 8, 12, 13]. For example, traditional blockchain applications require all nodes to perform identical on-chain computations, resulting in considerable overhead and latency. In contrast, *zkSNARK* enables off-chain computation, enabling nodes to validate only lightweight proofs to detect illegal state transitions. Additionally, its zero-knowledge attribute ensures confidential transactions. On platforms like Zcash [7] and Pinocchio Coin [8], although transaction details and user identities remain hidden, the integrity of transactions is verifiable, ensuring both security and privacy.

2.2 The *zkSNARK* Protocol

Achieving such innovative functionality requires a sophisticated protocol. This paper briefly introduces PLONK, a forefront *zkSNARK* protocol, to facilitate discussions on acceleration, referencing the protocol's details in [14].

Most *zkSNARKs* consist of four main stages: setup, circuit design, arithmetization, and proof generation. The setup phase generates several vectors of elliptic curve points for commitment and verification. The circuit design phase translates the statement to be proven into an equivalent arithmetic circuit. During arithmetization, this circuit is transformed into several vectors of scalar values, known as circuit constraints. Finally, the prover uses the elliptic curve points to prove that its witness satisfies the circuit constraints without disclosing the witness itself. In *zkSNARKs*, the first three phases are specific to the protocol or application and are conducted once, being reusable thereafter. Conversely, the proof generation stage is mandatory for each new proof.

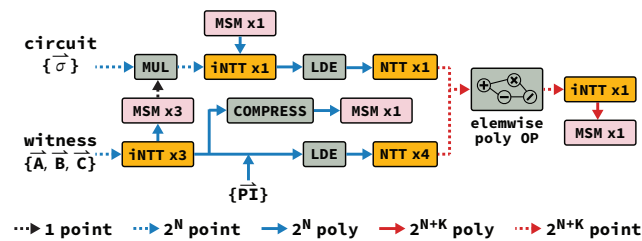


Figure 2. Workflow of PLONK-family *zkSNARK* protocols

Different *zkSNARK* protocols, while unique in the specifics of proof generation, typically follow the process outlined in Figure 2. Proving a witness that satisfies circuit constraints

involves an interaction between the witness and these constraints (i.e., element-wise polynomial operations). The outcome of this interaction is the quotient polynomial, albeit presented in point form. Subsequently, the polynomial variable (e.g., x in $f(x)$) is substituted with elliptic curve points, thus obtaining the polynomial's evaluation value (e.g., $f(x_0)$), through a process called MSM. This substitution requires the polynomial to be in its standard form, necessitating the use of inverse NTT ($iNTT$) to transfer from point form to polynomial form. The underlying mathematics ensures that the ability to compute this evaluation value implies knowledge of a witness that meets the circuit constraints, thus enabling succinct and zero-knowledge proofs.

PLONK enhances *zkSNARKs* with two key advancements [14]. First, it commits the raw witness and circuit constraints, enabling a universal setup phase for different applications. Second, PLONK supports more complex statements by permitting more sets of witness and circuit constraints. This expansion raises the quotient polynomial's degree, necessitating an extension of the original polynomials to a higher degree for computation, known as the low degree extension (LDE). LDE transforms the witness and circuit constraints from point form to polynomial form via $iNTT$, pads the polynomial to a higher degree, and then converts back to point form with NTT for further processing.

2.3 Number Theoretic Transform

NTT [1] is a specialized form of the Discrete Fourier Transform (DFT) for finite fields of integers. Unlike DFT, which uses $e^{-2\pi j/N}$ as its root of unity, NTT employs a primitive N_{th} root of unity, ψ , such that $\psi^N \equiv 1 \pmod{p}$, where $p = kN + 1$ is prime. The NTT formula, $X_k = \sum_{n=0}^{N-1} x_n \psi_{N,p}^{nk} \pmod{p}$, transforms input x_n into output X_k , where $\psi_{N,p}$ is the chosen root in $\mathbb{Z}_p$. Compared to DFT, NTT introduces challenges such as handling big integers, employing modular arithmetic, and requiring for pre-computed primitive roots.

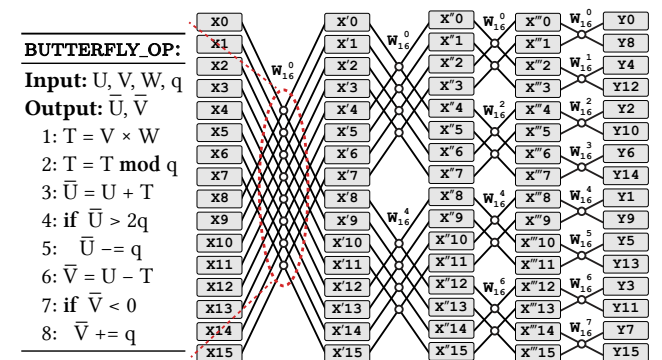


Figure 3. Illustration of 16-point Cooley-Tukey NTT

The Cooley-Tukey (CT) algorithm [6] and Gentleman-Sande (GS) algorithm [16], renowned for its efficiency in

Fast Fourier Transform, also stands out as the most effective algorithm for NTT. Figure 3 depicts the data access patterns of a 16-point CT-NTT. Each iteration executes 8 butterfly operations, and each butterfly operation in stage i accesses two elements with a stride of 2^{n-i} , producing two outputs for the next stage. The formula is $A' = A + B \times \psi^k N$ and $B' = A - B \times \psi^k N$, where A and B are the inputs, A' and B' are the outputs, and ψ_N^k signifies the root specific to the operation's stage and index. The GS algorithm, another NTT method, mirrors the CT algorithm but utilizes increasing strides (i.e., a stride of 2^i). The varying strides across stages lead to permutation data access patterns, incurring significant communication overhead in multi-GPU systems.

2.4 Motivation: Why multi-GPU NTT is necessary

When evaluating the execution time of NTT alone, multi-GPU acceleration might seem unnecessary due to potential inter-GPU communication latencies. However, within the context of end-to-end proof generation, multi-GPU NTT proves advantageous. The outputs of NTT serve as inputs for MSM, the most time-consuming component of zkSNARKs. Since MSM is accelerated with multiple GPUs, the outputs of NTT must be distributed across multiple GPUs, regardless of whether NTT is executed on one or several GPUs.

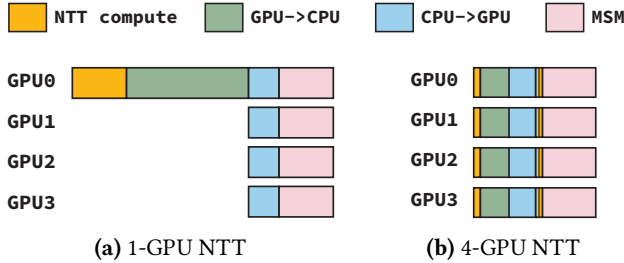


Figure 4. Comparison of single-GPU and multi-GPU NTT

Thus, counterintuitively, multi-GPU NTT actually reduces communication time in end-to-end proof generation. When NTT is executed on a single GPU, the output is transferred to the CPU through a single PCIe (Peripheral Component Interconnect Express) slot and then distributed to multiple GPUs via several PCIe slots, following a one-to-many communication schema (Figure 4a). As the PCIe slots' bandwidth is independent, other slots remain idle during the GPU→CPU transfer. In contrast, executing NTT on multiple GPUs allows for concurrent data distribution across the GPUs, as shown in Figure 4b. This approach enhances PCIe slot utilization and thus reduces the GPU→CPU data transfer time proportionally to the number of GPUs involved.

Why not assign one NTT per GPU? Allocating an independent NTT to each GPU can also fully utilize the PCIe slots. However, in most zkSNARK protocols, the number of NTTs that can be concurrently executed is much fewer than the number of GPUs in multi-GPU systems. For example, in

the zkSNARK protocols shown in Figure 2, an 8-GPU system only achieves a PCIe bandwidth utilization rate of 0.31, indicating considerable inefficiency.

3 Decompose NTT for Multi-GPU Systems

An N -point NTT is typically conceptualized as $\log(N)$ stages, each with a data access pattern determined by its index and the data layout. Thus, optimizing each NTT stage across all levels of the multi-GPU hierarchy is crucial. This section introduces a novel interpretation of the NTT algorithm, tailored to the hierarchical structure of multi-GPU systems: each level consists of subunits with dedicated memory; the NTT task is decomposed into smaller subtasks, necessitating communication between them.

This section discusses the decomposition of NTT with specific examples to aid understanding, but the methodologies are applicable for any configurations.

3.1 NTT Decomposition via Data Transpose

For a 16-point CT-NTT executed on four compute units, each compute unit can hold four points, a decomposition is depicted in Figure 5. Specifically, UNIT0 initially manages the points $\{X_0, X_4, X_8, X_{12}\}$ rather than two adjacent pairs $\{X_0$ and X_8, X_4 and $X_{12}\}$. This arrangement ensures that after the first round, UNIT0 holds $\{X'_0, X'_4, X'_8, X'_{12}\}$, matching the point pairs required for the second round. It eliminates the need for inter-unit communication or synchronization between the first and second rounds. After the second round, all-to-all communication among the units enables UNIT0 to obtain $\{X''_0, X''_4, X''_8, X''_{12}\}$, enabling independent execution of the third and fourth rounds of the NTT. Figure 6a abstracts away specific primitive roots, focusing on the algorithm's structure. It represents decomposition in a matrix format, where each subunit manages a column, synchronizes and exchanges data, then processes a row. For GS-NTT, the procedures reverse: each subunit handles a row, synchronizes and exchanges data, then processes a column.

While the size of an NTT must not exceed the resource capacity of a unit, the number of NTTs need not align with the number of units, as subunits can sequentially process multiple NTTs. Figure 6b shows an example of 64-point CT-NTT. In the column stage, with 16 columns, each thread processes four columns.

3.2 Recursive Decomposition

In the example in Figure 6b, resource constraints occur at the row stage, where each subunit processes a 16-point NTT. This issue can be addressed by applying further decomposition recursively, treating each row as a distinct matrix. For instance, the first row can be further decomposed, as shown in Figure 6a. Notably, the recursive decomposition is highly flexible. For example, a 64-point NTT can be decomposed in various ways, such as $4 \times (4 \times 4)$, $(4 \times 4) \times 4$, or $(2 \times 4) \times (2 \times 4)$.

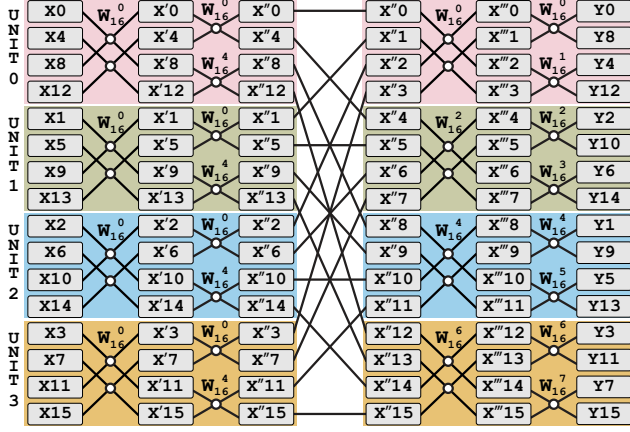


Figure 5. Decompose a 16-point NTT into four 4-point NTTs

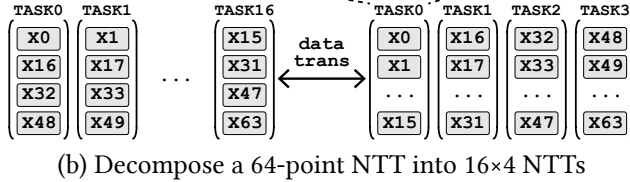
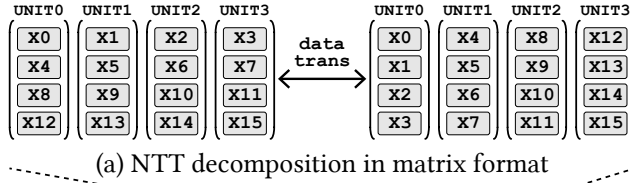


Figure 6. Illustration of NTT Decomposition via Matrix

3.3 Mix of Decomposition and Non-Decomposition

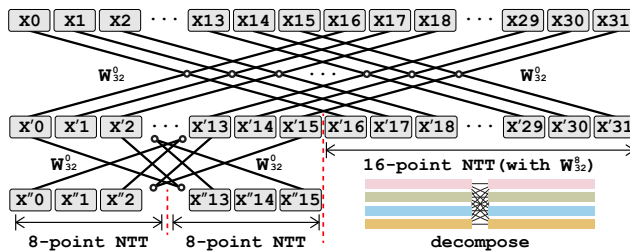


Figure 7. Mix of Decomposition and Non-Decomposition

The decomposition can be combined with standard NTT rounds. Figure 7 illustrates a 32-point CT-NTT example, where an initial conventional NTT round produces $X'_0, X'_1, \dots, X'_{31}$. The outputs are then split into two 16-point NTTs: $\{X'_0, \dots, X'_{15}\}$ and $\{X'_{16}, \dots, X'_{31}\}$. Each 16-point segment may either proceed with a standard NTT round or undergo further decomposition. For instance, the segment with W_{32}^0 might continue with a conventional round, while

the segment with W_{32}^8 could opt for decomposition. A conventional NTT round is equivalent to decomposing an N -point NTT into $2 \times \frac{N}{2}$ NTTs. It is explicitly discussed for its flexibility and potential to enhance various optimizations. For example, if the NTT size slightly exceeds the subunit's capacity, it may be optimal to execute a standard NTT round followed by each subunit processing two separate NTTs.

Algorithm 1 Decompose NTT for Multi-GPU Systems

```

1: function DECOMPOSE_NTT( $X, N$ )
2:   for  $i = 0$  to  $\log(N_0)$  do
3:      $X = \text{BUTTERFLY\_OP}(X, i)$   $\triangleright$  a normal NTT round
4:   if  $N_0 < N$  then
5:      $N' = N/N_0$ 
6:     for  $i = 0$  to  $N_0$  do  $\triangleright$  execute  $N_0$   $N'$ -point NTTs
7:        $X' = X[i \times N' : (i+1) \times N']$ 
8:        $N_1, N_2 = \text{FACTOR}(N')$   $\triangleright N_1 \times N_2 = N'$ 
9:       map  $N_1$  DECOMPOSE_NTT( $X', N_2$ ) to  $K$  subunits
10:      ALL_TO_ALL_COMMUNICATION( $X$ )
11:      map  $N_2$  DECOMPOSE_NTT( $X', N_1$ ) to  $K$  subunits
12:   end function

```

The pseudocode for the NTT decomposition algorithm is outlined in Algorithm 1. It begins with $\log(N_0)$ rounds of the conventional NTT, followed by the decomposition of each of the $\frac{N}{N_0}$ -point NTTs into $N_1 \times N_2$ NTTs, upon which the algorithm is recursively applied. Algorithm 1 omits specific primitive roots to focus on the high-level algorithm. Each $\frac{N}{N_0}$ -point NTT (Line 6) utilizes distinct primitive roots starting with $W_N^{N/(2N_0)}$, as illustrated in Figure 7. Subsequently, each $\frac{N}{N_0}$ -point NTT is decomposed. The N_2 -point NTT at Line 9 uses the same primitive roots, while the N_1 -point NTT at Line 11 employs different primitive roots, as demonstrated in Figure 5.

3.4 Comparison with 4-step NTT Algorithm

Algorithm 1 and the 4-step NTT share a similar structural framework. However, the 4-step NTT includes element-wise multiplication with a twiddle factor table between its column and row stages. It allows the use of the same primitive roots across smaller NTTs, improving memory access efficiency when these smaller NTTs are executed on the same core. Nevertheless, accessing $O(N)$ data elements is still required during the twiddle factor multiplication stage.

On the other hand, *UniNTT* is specifically designed for GPU architectures, where different rows are processed by distinct physical units, each equipped with dedicated memory resources. Using identical primitive roots across various rows results in redundant data replication across the memory resources of distinct units. Thus, *UniNTT*'s decomposition strategy assigns different primitive roots to each row, eliminating the need for element-wise multiplication with the

twiddle factor table. It ensures an overhead-free decomposition, such that the computation remains equivalent to the original NTT, regardless of the number of decompositions. This is a notable improvement. For comparison, the four-step NTT incurs a $\frac{4}{24} = 16.7\%$ computational overhead for a 2^{24} -point NTT executed on multi-GPU systems, which requires four decompositions as shown in Figure 1. Given the substantial cost associated with big integer modular multiplication, eliminating this overhead is crucial for enabling aggressive decomposition to allow each level of the multi-GPU hierarchy to perform the same NTT computations with different input sizes.

Moreover, *UniNTT*'s decomposition offers enhanced flexibility compared to the four-step NTT. In addition to matrix-style decomposition of NTTs, it also supports combining normal butterfly NTT rounds with matrix-style decomposition. This enhanced flexibility facilitates potential optimizations.

In summary, while the algorithms may seem similar, they stem from fundamentally different design philosophies. The 4-step NTT is optimized for sequential processors like CPUs, whereas *UniNTT* is designed for massively parallel architectures such as GPUs.

4 Optimize on Abstracted Hardware Model

Overhead-free decomposition enables the same NTT computations across all levels of the multi-GPU hierarchy, albeit at varying scales. *UniNTT* abstracts these levels into two components: subunits for NTT execution and memory for data exchange. It allows for optimization designs based on the abstracted hardware model rather than individual levels.

4.1 Primitive Root Reuse and Prefetch

Traditional NTT implementations require data exchange in each round, typically through fast memory such as shared memory within thread blocks. *UniNTT* reduces the data exchange to just once at each level of the multi-GPU hierarchy through aggressive decomposition. It eliminates the need to occupy fast memory for data exchange during column and row stages, enabling more effective and adaptable data caching strategies. In the column stage, data reuse for primitive roots proves efficient, as all subunits access identical primitive roots. This holds even when the column-stage NTT is decomposed from a row-stage NTT: though each unit has unique primitive roots compared to its peers, its subunits consistently utilize the same primitive roots. Thus, subunits can collaboratively load primitive roots into memory during the column stage, significantly minimizing accesses to slower memory. Conversely, in the row stage, each subunit uses distinct primitive roots, preventing data reuse. The available memory resources are instead allocated for prefetching these primitive roots, thus facilitating overlap between computations and memory operations.

4.2 Input Data Layout

The data layout in recursive NTT is complex, extending beyond simple row-major or column-major formats. In ZKP, NTT inputs are typically stored in a row-major layout, and at the GPU level, matrices adhere to this format. However, when further subdivided (treated as individual matrices), each row may follow a column-major layout. Moreover, data layouts may transform after NTT computation, as depicted in Figure 3, where a row-major matrix converts to a column-major matrix following a CT-NTT. In GPU programming, n -dimensional arrays are typically stored as one-dimensional arrays, with indices manually computed for interpretation. While this supports complex indexing, inappropriate data layouts can severely impact performance. Data layout management is critical for optimizing inter-GPU communication, for example, enhancing coalesced access to device memory, and preventing bank conflicts in shared memory. Furthermore, layouts should be systematically considered, as the layout at one level may influence another.

4.3 Multiplication Elimination

As depicted in Figure 3, at stage i , $\frac{N}{2^i}$ butterfly operations use the primitive root $W_N^0 = 1$, thereby omitting the computationally expensive big integer modular multiplications with the primitive root, a method referred to as multiplication elimination. This optimization provides marginal benefits in contexts such as FFTs with floating-point numbers and NTTs using small moduli (e.g., 64-bit in fully homomorphic encryption), where multiplication and addition have comparable costs. However, in ZKPs involving big integers, modular multiplication is notably more costly due to its $O(N^2)$ time complexity.

Algorithm 2 Decomposition with Multiplication Eliminate

```

1: function DECOMPOSE_NTT( $X, N, can\_eliminate$ )
2:   if  $N < N_0$  then  $\triangleright$  Threshold of decomposition
3:     for  $i = 0$  to  $\log N$  do
4:        $X = \text{BUTTERFLY\_OP}(X, i)$   $\triangleright$  a normal NTT round
5:     if  $can\_eliminate$  then  $\triangleright$  go through elimination
6:        $X = \text{BUTTERFLY\_NO\_MUL}(X, i)$   $\triangleright$  a normal NTT
        round without primitive root multiplication
7:       DECOMPOSE_NTT( $X[0 : N/2], N/2, \text{true}$ )
8:       DECOMPOSE_NTT( $X[N/2 : N], N/2, \text{false}$ )
9:     else  $\triangleright$  go through decomposition
10:       $N_1, N_2 = \text{FACTOR}(N)$   $\triangleright N_1 \times N_2 = N$ 
11:      map  $N_1$  DECOMPOSE_NTT( $X, N_2, \text{false}$ ) to  $K$  subunits
12:      ALL_TO_ALL_COMMUNICATION( $X$ )
13:      map  $N_2$  DECOMPOSE_NTT( $X, N_1, \text{false}$ ) to  $K$  subunits
14: end function

```

The revised decomposed NTT algorithm is detailed in Algorithm 2. If the N -point NTT uses W_N^0 , a standard NTT round is executed without multiplying by W_N^0 . Subsequently,

the first $\frac{N}{2}$ -point NTT employs W_N^0 , while the second starts with W_N^1 . In contrast, if the N -point NTT does not utilize W_N^0 , it undergoes decomposition to reduce memory accesses. It not only reduces computation but also produces two kernels that are memory-intensive and computation-intensive, respectively. Leveraging GPU capabilities for concurrent kernel execution, it optimizes the utilization of memory and computational resources.

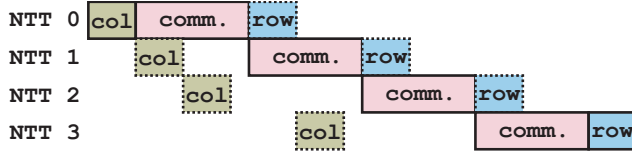


Figure 8. Pipelining with the shortest execution path

4.4 Shortest Path Pipelining

The data transpose stage often represents a bottleneck, as it occurs in slower memory. For example, in multi-GPU systems, while column and row stages are processed internally on each GPU, inter-GPU exchanges must pass through host memory. There is potential to overlap data exchange with computation. Specifically, a unit often manages multiple independent NTTs, each decomposed into three stages. The execution of the column/row stage of one NTT can be overlapped with the data transpose stage of another, effectively hiding latency. However, simply enabling concurrent execution does not ensure optimal latency hidden. The key to effective latency hidden lies in minimizing the critical path length. Figure 8 shows an example with 4 NTTs, where the critical path includes the first column-stage NTT, the communications of the 4 NTTs, and the final row-stage NTT. To minimize total execution time, the first data communication should be initiated as early as possible. Thus, only the first NTT starts initially, while the other three are delayed.

5 Systematically Deploy Optimizations to Multi-GPU System

This section discusses how optimizations, designed based on the abstract hardware model, are applied to multi-GPU systems. It uses a case study of executing a 2^{28} -point NTT with 256-bit scalars on eight PCIe-connected Nvidia GPUs to facilitate the discussion, although the concepts are applicable to other setups.

5.1 Multi-GPU System

As depicted in Figure 2, generating a proof involves one or several independent NTTs. The assumption of PCIe interconnection is based on the widespread availability of PCIe-connected GPUs and the straightforward adaptation of PCIe-based designs to NVLink.

Workload Decomposition. Despite having 8 GPUs, the NTT is not decomposed into $8 \times \frac{N}{8}$ NTTs, due to the inefficiency of executing 8-point NTTs on GPUs. Instead, it is decomposed into $\frac{N}{N_{TB}} \times N_{TB}$, where N_{TB} represents the scale of NTT handled by each thread block, set at 1024 for this instance. Then, the 1024 2^{18} -point NTTs are distributed across the 8 GPUs, with consecutive columns allocated to the same GPU to reduce communication overhead.

Primitive Root Reuse & Prefetch. For a given elliptic curve, primitive roots are constant. Thus, once loaded into host memory, they can be reused for all NTTs. The 2^{17} primitive roots require 4GB of memory. Given the host memory capacity of most multi-GPU systems, it is feasible and efficient to load all primitive roots at the start of the application and retain them for its entire runtime.

Input Data Layout. Due to the absence of direct shared memory among GPUs, the misalignment between data layouts and NTT stages necessitates all-to-all communication, the most latency-intensive process. Consequently, it is crucial to choose an NTT algorithm that matches the data layout: GS-NTT for row-major order and CT-NTT for column-major order. Importantly, the layout of the output is less critical since it is used for element-wise polynomial operations and MSM, which are adaptable to various data layouts, eliminating the need for output transpose.

Multiplication Elimination. The communication overhead associated with PCIe prohibits multiplication elimination at this level. With $N_{TB} = 1024$, the 13 ~ 18 NTT rounds do not benefit from multiplication elimination, affecting only 0.024% of butterfly operations with W^0 . It underscores the advantage of larger column sizes. If the NTT were decomposed into 1024×2^{18} NTTs, rounds 5 ~ 10 would be excluded from multiplication elimination, affecting 6.15% of the butterfly operations with W^0 .

Pipeline. At this level, pipelining primarily stems from the independent NTTs of proof generation, such as the 3 and 5 NTTs in Figure 2. To minimize latency, the shortest path pipeline optimization is employed. The asynchronous streams enable concurrent execution of computational and memory tasks, with synchronization managed through host-side APIs (Application Programming Interfaces). Given the extensive host memory in multi-GPU systems, integrating all independent NTTs into the pipeline is feasible.

5.2 Single GPU

According to the higher-level decomposition, the GPU level manages 2^{18} -point NTTs from the column stage and 1024-point NTTs from the row stage.

Workload Decomposition. Given resource constraints, a thread block can handle 1024-point NTTs but cannot support a 2^{18} -point NTT. Consequently, the 2^{18} -point NTT is decomposed into 512×512 NTTs.

Primitive Root Reuse & Prefetch. The column-stage NTTs share 2^{17} identical primitive roots, requiring only 4MB of memory for storage. It allows them to remain permanently in device memory, with *UniNTT* further locking them in L2 cache [34] to enhance performance. In contrast, row-stage NTTs employ distinct primitive roots. Each GPU processes 2^{15} instances, occupying a 0.5GB of memory per GPU. While most GPUs can maintain them in device memory permanently, GPUs with limited capacity may need to reallocate the 0.5GB for other tasks between NTT operations. Thus, prefetching becomes essential and can be efficiently implemented since each GPU manages 2^{15} 1024-point NTTs.

Input Data Layout. The data layout is derived from the multi-GPU level.

Multiplication Elimination. As the 1024-point NTTs are from the row stage, butterfly operations involving W^0 (totalling 1023) are minor compared to the overall number, making multiplication elimination unnecessary. In contrast, for the 2^{18} -point NTTs from the column stage, the 1024 instances execute $2^{28} - 2^{10}$ butterfly operations with W^0 , nearly matching the total count of $2^{28} - 1$. Consequently, multiplication elimination is applied until N reaches 512, with further optimizations managed at the thread block level.

Pipeline. At the multi-GPU system level, each NTT is decomposed into multiple NTTs, allowing even the single NTT in Figure 2 to leverage pipelining starting from the GPU level. A GPU executes 128 independent 2^{18} -point NTTs, with each further decomposed into 512 independent 512-point NTTs. When launching all these NTTs simultaneously, it involves a *load-compute-store-sync-load-compute-store* cycle, where extensive synchronization prevents effective latency hiding in device memory. Adhering to the shortest path pipeline design philosophy, *UniNTT* organizes these 128 independent NTTs into several groups. For example, by forming 8 groups, it enables 8 independent sequences, allowing overlap between computations and memory operations across groups. The number of groups is selected to balance kernel management overhead with overlap efficiency.

5.3 Thread Block

Due to higher-level decomposition, a thread block processes NTTs ranging in size from hundreds to 2^{11} points, covering both column and row stages.

Workload Decomposition. As each warp can process up to 64-point NTTs, utilizing 64-point NTTs as subtasks allows a single decomposition to efficiently manage 2^{12} -point NTTs, covering the necessary requirements.

Primitive Root Reuse & Prefetch. Primitive roots are collaboratively loaded into shared memory by warps for column-stage NTTs. For row-stage NTTs, primitive roots are prefetched from device memory into shared memory using asynchronous data copy operations.

Input Data Layout. Coalesced memory access optimizes memory bandwidth utilization by ideally having each warp

access a contiguous block of 32 32-bit elements. Although scalars are stored sequentially, each scalar comprises multiple uint32 elements, leading to non-coalesced access when each thread is assigned a big integer, due to a stride equal to the number of uint32 elements per scalar. To address this, the scalars required by the thread block are first collaboratively transferred to shared memory, from which each thread then fetches its scalar.

Multiplication Elimination. At the thread block level, multiplication elimination is applied solely in the first round of the NTT, as butterfly operations using W^0 beyond this point are minor, accounting for about 511K instances in this case. From the second level onward, it becomes necessary to differentiate between types of butterfly operations, which introduces inefficiencies due to the GPU's lack of branch prediction, even though it does not cause warp divergence.

Pipeline. The shared memory can accommodate several NTTs, each decomposed and organized into a pipeline as shown in Figure 8. Concurrent execution is facilitated by ping-pong buffers, while synchronization is achieved via hardware-level barriers. Typically, the pipeline incorporates 2 or 4 NTTs, depending on the shared memory's capacity and throughput.

5.4 Warp

According to higher-level decomposition, a warp manages NTTs ranging in size from dozens to 64 points.

Workload Decomposition. As a warp contains 32 threads, it can handle up to 64-point NTTs. Conceptually, the 64-point NTT can be regarded as a decomposition into a $2 \times 2 \times 2 \times 2 \times 2$ matrix, with each thread executes a 2-point NTT. In practice, the 64-point NTT is implemented directly, achieving the same effect as the conceptual decomposition. For NTTs smaller than 64 points, a warp executes multiple NTTs concurrently, and since each NTT requires identical computations, it incurs no warp divergence.

Primitive Root Reuse & Prefetch. The intense demand on registers prohibits explicit caching or prefetching of primitive roots at the warp level. Instead, threads access primitive roots directly from shared memory of the thread block level. Thanks to the shared memory broadcasting features in most GPUs, when multiple threads within a warp access identical shared memory addresses, data reuse naturally occurs.

Input Data Layout. Big integers not only hinder coalesced access to device memory but also induce bank conflicts in shared memory. For example, many elliptic curves employ 32-byte scalars (e.g., BN254, BLS12-377, and BLS12-381), equivalent to 8 uint32 elements. A common access pattern involves *thread0* reading words 0~7 and *thread1* reading words 8~15, resulting in an 8-way bank conflict. To mitigate this, *UniNTT* transposes inputs cached in shared memory into an element-major layout after loading from device memory and before each warp accesses them. The element-major layout sequentially stores the first, second, and subsequent words

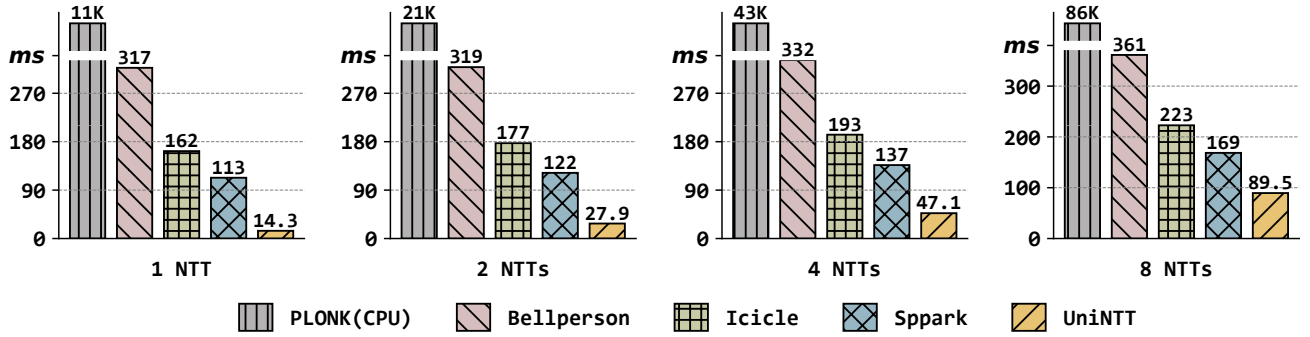


Figure 9. Execution time of the evaluated methods with different number of independent NTTs

of all integers. Although reorganizing the input data into element-major order at the application level can resolve data layout issues at both the thread block and warp levels, this approach may not be practical for all ZKP implementations. **Multiplication Elimination.** Multiplication elimination is stopped at the thread block level due to its negligible benefit. **Pipeline.** At the warp level, pipelines are not implemented because native data shuffle operations enable highly efficient intra-warp communication. Moreover, allowing each thread to perform multiple NTTs would require storing several rows or columns in registers, significantly increasing register pressure — a critical bottleneck in big integer arithmetic.

6 Evaluation

The evaluation comprises three parts. First, it assesses the overall performance regarding NTT operations, end-to-end zkSNARK workloads, and scalability on multi-GPU systems. Second, performance across various GPUs and elliptic curves is evaluated. Finally, it presents a breakdown analysis of the proposed optimizations.

Experiments utilize the BLS12-381 elliptic curve and are conducted on a multi-GPU system equipped with 8 Nvidia RTX 4090 GPUs connected via PCIe 4.0 $\times 16$. Benchmarks include GPU-based NTTs from *Bellperson* [12], *Icicle* [24], and *Sppark* [29], alongside a CPU-based NTT from *PLONK* [39] for comparison. In evaluations, the best GPU baseline for each benchmark is referred to as the SOTA (state-of-the-art). *UniNTT* is the only NTT implementation featuring multi-GPU acceleration. Other GPU implementations are extended to multi-GPU versions by assigning an independent NTT to each GPU. For a fair comparison, the time required to distribute NTT outputs across GPUs is measured, providing a more accurate reflection of the proof generation in practice.

6.1 Overall Performance

Figure 9 shows the performance of various NTT implementations across different numbers of concurrent NTTs, ranging from 1 to 8. *UniNTT* achieves an average speedup of 4.26 $\times$ compared to the SOTA, attributable to multi-GPU support

and comprehensive optimizations. Furthermore, all GPU-based implementations surpass CPU counterparts by at least 117.0 $\times$, highlighting the efficiency of GPU-accelerated NTT. Notably, *UniNTT* achieves a 7.88 $\times$ speedup over SOTA when executing a single NTT. As the number of concurrent NTTs increases to 8, the speedup decreases to 1.89 $\times$. According to Figure 2, the typical number of independent NTTs in real ZKP protocols is one, making *UniNTT*'s speedup near the higher one in practical scenarios.

While *UniNTT*'s execution time scales with the number of NTTs, the baselines remain nearly constant. It does not indicate *UniNTT*'s inefficiency when handling multiple NTTs; rather, it underscores the limitations of other methods that lack true multi-GPU support and thus underperform when the number of NTTs is fewer than the available GPUs.

6.2 End-to-End Proof Generation

Table 1 presents the end-to-end proof generation times for various zkSNARK workloads, including digital currencies (i.e., Merkle-Tree [4]) and verifiable machine learning (i.e., Otti-SGD [3] and Zen_{acc}-LeNet [11]). The data focuses exclusively on core proof generation steps depicted in Figure 2, excluding auxiliary operations such as transcript writing. Proof generation involves NTT, polynomial arithmetic, and MSM, all utilizing multi-GPU acceleration. Polynomial arithmetic exploits its inherently embarrassingly parallel nature, while MSM is extended from *Sppark*'s implementation to enable multi-GPU execution.

Table 1. Performance of end-to-end workloads (in ms)

	Otti-SGD	Merkle-Tree	Zen _{acc}
Size	6,968,254	12,648,260	77,689,757
CPU	790.3K	1615.0K	5393.2K
Bellperson	800.0 (1.0 $\times$)	1631.1 (1.0 $\times$)	6266.4 (1.0 $\times$)
Icicle	515.6 (1.6 $\times$)	1097.9 (1.5 $\times$)	3948.6 (1.6 $\times$)
Sppark	422.6 (1.9 $\times$)	852.5 (1.9 $\times$)	3100.2 (2.0 $\times$)
UniNTT	220.3 (3.6 $\times$)	427.7 (3.8 $\times$)	1538.1 (4.1 $\times$)

GPU acceleration of NTT does not modify the ZKP protocol, thus maintaining its security integrity and generating proofs identical to those on CPUs. These proofs are verifiable through *PLONK*. Due to the succinct nature of *zkSNARKs*, both proof size and verification complexity remain at $O(1)$, with proofs at 2.53KB and verification times of 44.6ms on CPUs, independent of the complexity of proving statements.

UniNTT yields an average speedup of 3710 $\times$ over the CPU version for end-to-end proof generation, which underscores the efficacy of multi-GPU acceleration for *zkSNARK* proof generation. Selecting *Bellperson* as the GPU baseline, *UniNTT* achieves an average speedup of 3.84 $\times$, peaking at 4.07 $\times$. Compared with the SOTA, *UniNTT* achieves an average speedup of 1.98 $\times$, and up to 2.01 $\times$.

Notably, all methods utilize the same MSM implementation, accounting for 65.2% of the computational workload. Despite the NTT component constituting only 20.9% of total computation, *UniNTT* achieves a 3.84 $\times$ end-to-end speedup compared to the GPU baseline. When computational tasks are effectively accelerated on multi-GPU systems, the bottleneck shifts to the data transfer times required for distributing NTT outputs. This observation underscores the importance of multi-GPU support for NTT in ZKP; although NTT does not dominate computation time and multi-GPU acceleration does not significantly reduce NTT time, it enhances overall system efficiency by optimizing PCIe utilization.

6.3 Scalability

This section evaluates the scalability of different methods, focusing on both NTT itself and overall proof generation. With a single GPU, there is no communication overhead. Conversely, in multi-GPU setups, the NTT duration includes data transfer times, effectively capturing the costs linked to the permutation data access pattern of NTT.

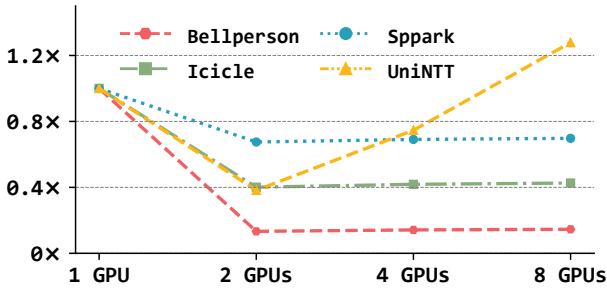


Figure 10. Scalability in terms of standalone NTT

Figure 10 illustrates the scalability of various methods in NTT times. Transitioning from a single-GPU to a multi-GPU configuration results in a significant performance decline, primarily due to the incurred communication overhead. Baseline methods, lacking multi-GPU support for NTT, show no performance improvement when additional GPUs are used

for a single NTT task. Conversely, *UniNTT*'s performance increases with the addition of GPUs in the multi-GPU setup. Despite communication costs, *UniNTT*'s performance with eight GPUs exceeds that of a single GPU.

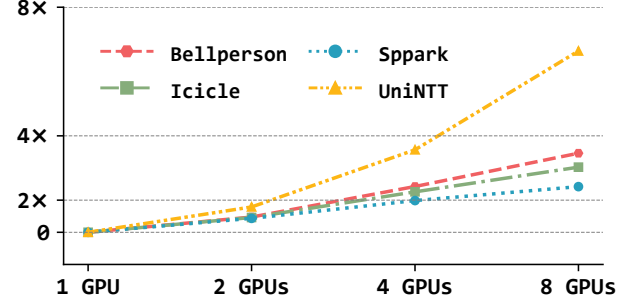


Figure 11. Scalability in terms of proof generation

However, this does not imply that multi-GPU acceleration is ineffective. As depicted in Figure 11, the scalability of end-to-end proof generation is robust, even for baselines that only support allocating a single NTT to each GPU. With multi-GPU support for NTT, *UniNTT* achieves a 6.64 $\times$ speedup on an 8-GPU system compared to single-GPU setups, markedly surpassing the baselines. It underscores the pivotal role of multi-GPU NTT in enhancing the efficiency of multi-GPU systems for ZKP acceleration.

Notably, scalability does not directly correlate with absolute performance. For instance, *UniNTT* exhibits suboptimal scalability in NTT time on a 2-GPU system, yet it delivers significantly higher absolute performance compared to others. Moreover, *Sppark* exhibits the poorest scalability in end-to-end proof generation but outperforms other baselines in terms of absolute execution time on an 8-GPU system, but still requires 1.92 $\times$ the execution time of *UniNTT*.

6.4 Performance on Various GPUs and Curves

This section evaluates *UniNTT*'s performance across various GPUs and elliptic curves. The GPUs include the Nvidia H100, Nvidia A100, Nvidia RTX 4090, and Nvidia RTX 4060Ti, using a single GPU configuration. The elliptic curves analyzed are BN254, BLS12-377, BLS12-381, and MNT4753. To prevent memory overflow on lower-end GPUs, the NTT size is fixed at 2^{26} . Throughput serves as the performance metric, facilitating direct comparison with GPU specifications.

UniNTT shows robust performance across various GPUs and elliptic curves, as shown in Figure 12. The performance primarily depends on the arithmetic throughput when GPU memory bandwidth is adequate. However, on GPUs with limited memory bandwidth, such as the Nvidia RTX 4060Ti, this factor significantly affects performance. For elliptic curves such as BN254, BLS12-377, and BLS12-381, with scalar bit lengths around 256 bits, NTT performance is similar. In contrast, MNT4753, with a 753-bit scalar, requires about nine

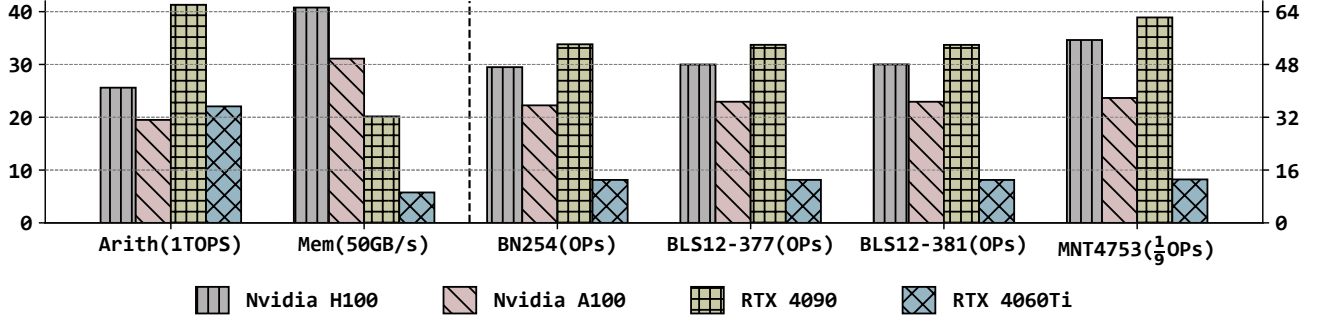


Figure 12. Performance on various GPUs and elliptic curves. The left part compares the hardware resources of the four GPUs, corresponding to the left y-axis. The right part compares the throughput, corresponding to the right y-axis. The unit of the y-axis is specified in parentheses under each group.

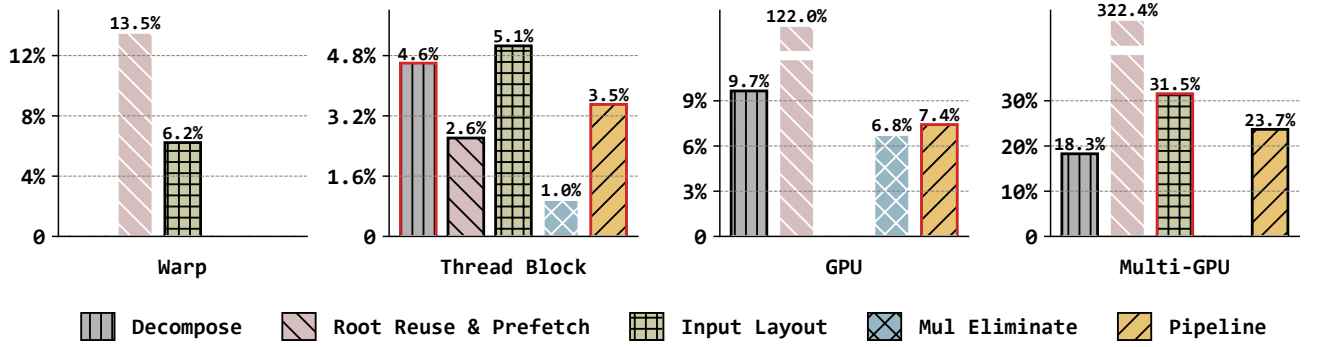


Figure 13. Performance breakdown of the optimizations applied at different levels. White borders highlight common optimizations in NTT implementations, black borders signal optimizations enabled by *UniNTT*'s overhead-free decomposition method, and red borders denote optimizations considered level-specific but proven effective across various levels.

times the theoretical arithmetic operations compared to these curves. Yet, *UniNTT* only requires 8.27× the execution time on average and 7.81× on Nvidia H100 GPUs. This contrasts with MSM, where MNT4753's actual execution time exceeds theoretical expectations due to complex elliptic curve arithmetic and increased register pressure [25]. NTT is more memory-intensive. MNT4753's memory access is only three times that of other elliptic curves, leading to lower execution times than theoretically predicted arithmetic demands.

6.5 Breakdowns of Optimizations

Figure 13 presents a breakdown evaluation of *UniNTT*'s optimizations, using a 2^{28} -point NTT from the BLS12-381 curve on an Nvidia H100 GPU. The baseline is an unoptimized NTT kernel. The speedups are classified into three categories: optimizations employed in most NTT implementations (white borders), optimizations enabled by *UniNTT*'s overhead-free decomposition method (black borders), and optimizations considered level-specific but found effective across multiple levels (red borders).

UniNTT's overhead-free decomposition not only enables a unified design of optimizations but also directly boosts performance in two ways. Firstly, unlike the four-step NTT, it removes the necessity to multiply the twiddle factor table, resulting in a 6.8% speedup at the GPU level. Secondly, it enables aggressive decomposition at the thread block level, which changes shared memory access to register file shuffling, thereby achieving a 4.6% speedup. Additionally, *UniNTT* avoids the inefficient 8-point NTT at the multi-GPU level, achieving a 18.3% speedup.

UniNTT selects algorithms based on the current data layout to avoid inter-GPU communication and uncoalesced device memory access, achieving a 31.5% speedup at the multi-GPU and GPU levels. Optimizing memory access pattern provides a modest improvement at the thread block level, where it is already relatively efficient. *UniNTT* achieves a 5.1% speedup by preventing bank conflicts at the warp level. Bank conflicts are common and severe in big integer arithmetic, and modifying the data layout from an end-to-end scope may not always be feasible. *UniNTT* addresses this challenge by implementing data transposition at the thread

block level, an optimization beneficial to other GPU kernels handling big integer arithmetic.

Existing implementations adopt root reuse and prefetch but fail to apply it at the thread block level, as shared memory is occupied by intermediate results. In contrast, *UniNTT*'s decomposition method employs shared memory exclusively for data exchange, enabling the use of shared memory for root reuse and prefetch. Additionally, the pipeline method, typically specific to multi-GPU contexts, is effectively implemented at both the thread block and GPU levels in *UniNTT*'s unified design, achieving extra speedups of 3.5% and 7.4% respectively. For multiplication elimination, an accumulative speedup of 7.8% is achieved, aligning closely with theoretical expectations. This result confirms the minimal conflict between multiplication elimination and decomposition.

In summary, the overhead-free decomposition not only enables previously infeasible optimizations but also supports a unified optimization design approach. Experiments demonstrate that optimizations previously specific to one level often benefit other levels as well.

7 Related Work

Previous studies have proposed many novel optimizations for NTT in ZKP on various hardware platforms. For GPUs, GZKP [31] improves performance through cache-friendly memory access patterns. Ni et al. [32] suggests padding integers and rearranging data in shared memory to reduce bank conflicts. Merged-NTT [35] achieves a $1.17\times$ speedup on GPUs by forgoing the 4-step NTT. On FPGAs, Zhao et al. design K -radix NTT and data-reordering techniques to enhance parallelism and pipeline efficiency [42]. Proteus [23] offers a parametric design for bandwidth-efficient, pipelined Radix-2 NTT architectures. Hardcaml, the ZPrize competition winner, utilizes 4-step NTT to enhance data locality [37]. For ASICs (Application-Specific Integrated Circuits), PipeZK [41] integrates 4-step NTT with an innovative pipeline for efficient strided accesses. Moreover, DIZK [40] employs 4-step NTT to distribute computations across CPU clusters. *UniNTT*, differing from these methods, accelerates NTT on multi-GPU systems through overhead-free decomposition and a unified design of optimizations.

In addition to ZKP, NTT is employed in several other domains such as integer multiplication [5], post-quantum cryptography, and fully homomorphic encryption (FHE), all benefiting from GPU acceleration. For many post-quantum cryptographic algorithms [2, 15, 28], the NTT size is sufficiently small to fit in shared memory, allowing processing by a single thread block. Numerous GPU optimizations have been tailored for FHE, which often involves 64-bit, 2^{16} -point NTTs. Kim et al. [26] provide a comprehensive evaluation of optimizations such as batch size, high radix, and caching twiddle factors, leading to an efficient GPU implementation.

Ozerk et al. [36] observe that NTT sizes in FHE slightly exceed shared memory capacities and propose a hybrid strategy involving initial rounds in device memory before shifting to shared memory computations by each thread block. Additionally, various studies explore using tensor cores [9, 10], known for high arithmetic throughput, to accelerate NTT, bypassing low time complexity algorithms (e.g., CT-NTT or GS-NTT) to leverage hardware acceleration and achieve regular data access patterns.

In addition to NTT design, big integer arithmetic significantly impacts performance. Numerous studies have explored GPU parallelization of big integer arithmetic [29, 33, 43]. Moreover, significant efforts have been made to optimize Montgomery modular multiplication on GPUs [27, 38]. *UniNTT*, orthogonal to these works, directly adopts the big integer arithmetic implementation from *Sppark*.

8 Discussion

The proposed optimization routine, while primarily designed to accelerate NTT in ZKP, may also facilitate architecture-aware optimization for other algorithms on multi-GPU systems. **First**, it ensures that all levels of the multi-GPU hierarchy execute identical computations. For instance, *UniNTT* employs a flexible, overhead-free NTT decomposition approach, allowing each hierarchy level to execute identical NTT computations with varying input sizes. **Second**, each level of the multi-GPU hierarchy is abstracted into a unified hardware model. It facilitates unified analysis and optimization of the algorithms, applicable to both a single warp and multi-GPU systems. In the context of NTT, where data exchange is critical, the hardware is abstracted into two main components: subunits for NTT execution and memory for data exchange, intentionally omitting specific hardware types. This abstraction allows *UniNTT* to design optimizations for the abstracted hardware model rather than for each specific hardware architecture. **Third**, these optimizations are systematically adapted and applied at each level of the multi-GPU hierarchy to achieve architecture-aware optimization. Notably, optimizations initially designed for one level often prove beneficial at others, as evidenced by *UniNTT* (i.e., Figure 13). Identifying new applications for existing optimizations represents a significant advantage of the proposed optimization routine.

9 Conclusion

In this study, we underscore the necessity of multi-GPU support for NTT in ZKP, which inspires the development of *UniNTT*. *UniNTT* employs an overhead-free decomposition method, enabling aggressive decomposition so that all levels of the multi-GPU hierarchy execute the same NTT computations at varying scales. It allows to design NTT optimizations based on an abstract hardware model, rather than specific levels. *UniNTT* integrates several optimizations, including

customized decomposition, primitive root reuse, prefetching, input data layout optimization, multiplication elimination, and pipelining. These optimizations are systematically tailored and deployed across the multi-GPU system, enhancing performance. Compared to SOTA implementations, *UniNTT* achieves an average speedup of 4.26 $\times$. In end-to-end proof generation, it achieves a 6.64 $\times$ speedup on an 8-GPU system over single-GPU setups, highlighting the efficiency of multi-GPU systems in accelerating ZKPs.

As future work, we plan to extend *UniNTT* to enhance NTT computations across various domains, such as homomorphic encryption. We are also interested in exploring other optimizations based on the abstracted hardware model.

Acknowledgments

We appreciate our shepherd and the anonymous reviewers for their constructive comments and suggestions. This work is supported by the Natural Science Foundation of China grant No. 62432005, No. 62402282, and No. 62372272, Shandong Provincial Natural Science Foundation No. ZR2024QF237 and No. ZR2024MF099, Department of Science & Technology of Shandong Province grant No. SYS202201, Quan Cheng Laboratory grant No. QCLZD202302, Taishan Scholars Program No. tsqn202211281.

References

- [1] Ramesh C Agarwal and C Sidney Burrus. 1975. Number theoretic transforms to implement fast digital convolution. *Proc. IEEE* 63, 4 (1975), 550–560.
- [2] Sedat Akleyek, Özgür Dağdelen, and Zaliha Yüce Tok. 2015. On the efficiency of polynomial multiplication for lattice-based cryptography on GPUs using CUDA. In *International Conference on Cryptography and Information Security in the Balkans*. Springer, 155–168.
- [3] Sebastian Angel, Andrew J Blumberg, Eleftherios Ioannidis, and Jess Woods. 2022. Efficient Representation of Numerical Optimization Problems for SNARKs. In *31st USENIX Security Symposium*.
- [4] Aritra Banerjee, Michael Clear, and Hitesh Tewari. 2020. Demystifying the Role of zk-SNARKs in Zcash. In *2020 IEEE conference on application, information and network security (AINS)*. IEEE, 12–19.
- [5] Boon-Chiao Chang, Bok-Min Goi, Raphael C-W Phan, and Wai-Kong Lee. 2018. Multiplying very large integer in GPU with pascal architecture. In *2018 IEEE Symposium on Computer Applications & Industrial Electronics (ISCAIE)*. IEEE, 401–405.
- [6] James W Cooley and John W Tukey. 1965. An algorithm for the machine calculation of complex Fourier series. *Mathematics of computation* 19, 90 (1965), 297–301.
- [7] ZCash Crop. 2024. Zcash is cash for the new age. <https://z.cash>.
- [8] George Danezis, Cedric Fournet, Markulf Kohlweiss, and Bryan Parno. 2013. Pinocchio coin: building zerocoin from a succinct pairing-based proof system. In *First ACM workshop on Language support for privacy-enhancing technologies*. 27–30.
- [9] Sultan Durrani, Muhammad Saad Chughtai, Mert Hidayetoglu, Rashid Tahir, Abdul Dakkak, Lawrence Rauchwerger, Fareed Zaffar, and Wenmei Hwu. 2021. Accelerating fourier and number theoretic transforms using tensor cores and warp shuffles. In *2021 30th International conference on parallel architectures and compilation techniques (PACT)*. IEEE, 345–355.
- [10] Shengyu Fan, Zhiwei Wang, Weizhi Xu, Rui Hou, Dan Meng, and Mingzhe Zhang. 2023. Tensorfhe: Achieving practical computation on encrypted data using gpgpu. In *2023 IEEE International Symposium on High-Performance Computer Architecture*. IEEE, 922–934.
- [11] Boyuan Feng, Lianke Qin, Zhenfei Zhang, Yufei Ding, and Shumo Chu. 2021. Zen: An optimizing compiler for verifiable, zero-knowledge neural network inferences. *Cryptology ePrint Archive* (2021).
- [12] Filecoin. 2024. bellperson: Gpu parallel acceleration for zk-snark. <https://github.com/filecoin-project/bellperson>.
- [13] Mina Foundation. 2024. GPU Groth16 prover (3x faster than CPU). <https://github.com/MinaProtocol/gpu-groth16-prover-3x>.
- [14] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. 2019. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive* (2019).
- [15] Yiwen Gao, Jia Xu, and Hongbing Wang. 2021. CuNH: Efficient GPU implementations of post-quantum KEM NewHope. *IEEE Transactions on Parallel and Distributed Systems* 33, 3 (2021), 551–568.
- [16] W Morven Gentleman and Gordon Sande. 1966. Fast fourier transforms: for fun and profit. In *Proceedings of the November 7–10, 1966, fall joint computer conference*. 563–578.
- [17] Craig Gentry and Daniel Wichs. 2011. Separating succinct non-interactive arguments from all falsifiable assumptions. In *43th annual ACM symposium on Theory of computing*. 99–108.
- [18] Oded Goldreich and Hugo Krawczyk. 1996. On the composition of zero-knowledge proof systems. *SIAM J. COMPUT* 25 (1996).
- [19] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. 1989. The Knowledge Complexity of Interactive Proof Systems. *SIAM J. COMPUT* 18 (1989).
- [20] Yinjie Gong, Yifei Jin, Yuchan Li, Ziyi Liu, and Zhiyi Zhu. 2022. Analysis and comparison of the main zero-knowledge proof scheme. In *2022 International Conference on Big Data, Information and Computer Network*. IEEE, 366–372.
- [21] Jens Groth. 2005. Non-interactive zero-knowledge arguments for voting. In *Applied Cryptography and Network Security: Third International Conference*. Springer, 467–482.
- [22] Jens Groth. 2016. On the size of pairing-based non-interactive arguments. In *35th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 305–326.
- [23] Florian Hirner, Ahmet Can Mert, and Sujoy Sinha Roy. 2023. PROTEUS: A Tool to generate pipelined Number Theoretic Transform Architectures for FHE and ZKP applications. *Cryptology ePrint Archive* (2023).
- [24] Icicle. 2024. a GPU Library for Zero-Knowledge Acceleration. <https://github.com/ingonyama-zk/icicle>.
- [25] Zhuoran Ji, Zhiyuan Zhang, Jiming Xu, and Lei Ju. 2024. Accelerating Multi-Scalar Multiplication for Efficient Zero Knowledge Proofs with Multi-GPU Systems. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 57–70.
- [26] Sangpyo Kim, Wonkyung Jung, Jaiyoung Park, and Jung Ho Ahn. 2020. Accelerating number theoretic transformations for bootstrappable homomorphic encryption on gpus. In *2020 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 264–275.
- [27] Karl Leboeuf, Roberto Muscedere, and Majid Ahmadi. 2013. A GPU implementation of the Montgomery multiplication algorithm for elliptic curve cryptography. In *2013 IEEE International Symposium on Circuits and Systems*. IEEE, 2593–2596.
- [28] Wai-Kong Lee, Sedat Akleyek, Denis Chee-Keong Wong, Wun-She Yap, Bok-Min Goi, and Seong-Oun Hwang. 2021. Parallel implementation of Nussbaumer algorithm and number theoretic transform on a GPU platform: application to qTESLA. *The Journal of Supercomputing* 77 (2021), 3289–3314.
- [29] Supranational LLC. 2024. Zero-knowledge template library. <https://github.com/supranational/spark>.
- [30] Tao Lu, Chengkun Wei, Ruijing Yu, Chaochao Chen, Wenjing Fang, Lei Wang, Zeke Wang, and Wenzhi Chen. 2022. Cuzk: Accelerating

- zero-knowledge proof with a faster parallel multi-scalar multiplication algorithm on gpus. *Cryptology ePrint Archive* (2022).
- [31] Weiliang Ma, Qian Xiong, Xuanhua Shi, Xiaosong Ma, Hai Jin, Haozhao Kuang, Mingyu Gao, Ye Zhang, Haichen Shen, and Weifang Hu. 2023. GZKP: A GPU Accelerated Zero-Knowledge Proof System. In *28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*.
 - [32] Ning Ni and Yongxin Zhu. 2023. Enabling zero knowledge proof by accelerating zk-SNARK kernels on GPU. *J. Parallel and Distrib. Comput.* 173 (2023), 20–31.
 - [33] NVIDIA. 2021. CGBN: CUDA Accelerated Multiple Precision Arithmetic using Cooperative Groups. <https://github.com/NVlabs/CGBN>.
 - [34] Nvidia. 2024. Device Memory L2 Access Management. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#device-memory-l2-access-management>.
 - [35] Ali Şah Özcan and Erkey Savaş. 2023. Two Algorithms for Fast GPU Implementation of NTT. *Cryptology ePrint Archive* (2023).
 - [36] Özgün Özerk, Can Elgezen, Ahmet Can Mert, Erdoğ Öztürk, and Erkey Savaş. 2022. Efficient number theoretic transform implementation on GPU for homomorphic encryption. *The Journal of Supercomputing* 78, 2 (2022), 2840–2872.
 - [37] Andy Ray, Ben Devlin, Fu Yong Quah, and Rahul Yesantharao. 2024. High performance, open source cryptographic solutions for large scale number theoretic transforms and multi-scalar multiplications in Hardcaml. https://github.com/fyquah/hardcaml_zprize.
 - [38] Nicolae Roşia, Virgil Cervesescu, and Mihai Togan. 2015. Efficient montgomery multiplication on GPUs. In *International Conference for Information Technology and Communications*. Springer.
 - [39] ZK-Garage Team. 2024. A pure Rust PLONK implementation using arkworks as a backend. <https://github.com/ZK-Garage/plonk>.
 - [40] Howard Wu, Wenting Zheng, Alessandro Chiesa, Raluca Ada Popa, and Ion Stoica. 2018. {DIZK}: A distributed zero knowledge proof system. In *27th USENIX Security Symposium*. 675–692.
 - [41] Ye Zhang, Shuo Wang, Xian Zhang, Jiangbin Dong, Xingzhong Mao, Fan Long, Cong Wang, Dong Zhou, Mingyu Gao, and Guangyu Sun. 2021. Pipezk: Accelerating zero-knowledge proof with a pipelined architecture. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture*. IEEE, 416–428.
 - [42] Haixu Zhao, Dong Ding, Feng Wang, Pengcheng Hua, Ning Wang, Qin Wu, and Zhilei Chai. 2022. Hardware acceleration of number theoretic transform for zk-SNARK. *Engineering Reports* (2022), e12639.
 - [43] Kaiyong Zhao and Xiaowen Chu. 2010. GPUMP: A multiple-precision integer library for GPUs. In *2010 10th IEEE International Conference on Computer and Information Technology*. IEEE, 1164–1168.